

Task 1

To Implement how to List Active Jobs, Display Status Information, Retrieve Job [Ds, Format Output, Update State Information, Test Job Listings

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int id;
    pid_t pid;
    char command[50];
    char state[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void update_jobs()
{
    int i;
    int status;

    for (i = 0; i < job_count; i++)
    {
        pid_t result = waitpid(
```

```

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result > 0)
        {
            strcpy(jobs[i].state, "Done");
        }
    }
}

void show_jobs()
{
    int i;

    update_jobs();

    printf("\nActive Jobs\n");
    printf("-----\n");

    if (job_count == 0)
    {
        printf("No jobs.\n");
        return;
    }
}

```

```

    }

    for (i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d  %-15s %s\n",
               jobs[i].id,
               jobs[i].pid,
               jobs[i].command,
               jobs[i].state);
    }

    printf("-----\n");
}

int main()
{
    int choice;

    while (1)
    {
        printf("\n1. Start Background Job\n");
        printf("2. List Jobs\n");
        printf("3. Exit\n");
        printf("Enter choice: ");

        scanf("%d", &choice);
    }
}

```

```
if (choice == 1)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        continue;
    }

    pid_t pid = fork();

    if (pid == 0)
    {
        execlp("sleep", "sleep", "10", NULL);

        perror("sleep");
        return 1;
    }
    else if (pid > 0)
    {
        jobs[job_count].id = job_count + 1;
        jobs[job_count].pid = pid;

        strcpy(jobs[job_count].command, "sleep 10");
        strcpy(jobs[job_count].state, "Running");
    }
}
```

```
        job_count++;

        printf("Started job [%d], PID = %d\n",
               job_count, pid);
    }
    else
    {
        perror("fork");
    }
}
else if (choice == 2)
{
    show_jobs();
}
else if (choice == 3)
{
    break;
}
else
{
    printf("Invalid choice.\n");
}
}

return 0;
}|
```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 9$ ./task1
```

```
1. Start Background Job
```

```
2. List Jobs
```

```
3. Exit
```

```
Enter choice: 1
```

```
Started job [1], PID = 1274
```

```
1. Start Background Job
```

```
2. List Jobs
```

```
3. Exit
```

```
Enter choice: 2
```

```
Active Jobs
```

```
-----  
[1] PID=1274  sleep 10          Running  
-----
```

```
1. Start Background Job
```

```
2. List Jobs
```

```
3. Exit
```

```
Enter choice: 3
```

```
geya@LenovoL480:~/OS/Skill/Week 9$ |
```

Task 2

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    int status;

    printf("Starting background process...\n");

    pid = fork();

    if (pid == 0)
    {
        printf("Child: PID = %d\n", getpid());

        sleep(3);

        printf("Child: process completed.\n");

        exit(0);
    }
}
```

```

if (pid > 0)
{
    printf("Parent: background process started.\n");
    printf("Parent: PID = %d\n", pid);

    sleep(1);

    printf("\nBringing process to foreground...\n");

    /*
     * Wait for the selected job
     */
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Foreground job completed.\n");
    }
}
else
{
    perror("fork");
}

return 0;
}|

```

Output:

```

geya@LenovoL480:~/OS/Skill/Week 9$ ./task2
Starting background process...
Parent: background process started.
Parent: PID = 1325
Child: PID = 1325

Bringing process to foreground...
Child: process completed.
Foreground job completed.
geya@LenovoL480:~/OS/Skill/Week 9$ |

```

Task 3

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid == 0)
    {
        /*
         * Create a new process group
         */
        setpgid(0, 0);

        printf("Child running. PID = %d\n", getpid());

        sleep(5);

        printf("Child completed.\n");
    }
}
```

```

        exit(0);
    }

    if (pid > 0)
    {
        /*
         Put child in its own process group
        */
        setpgid(pid, pid);

        sleep(1);

        printf("Giving terminal to child process...\n");

        /*
         Give terminal control to child group
        */
        tcsetpgrp(STDIN_FILENO, pid);

        /*
         Wait for child
        */
        waitpid(pid, &status, 0);

        /*
         Take terminal back

```

```

        tcsetpgrp(STDIN_FILENO, getpgrp());

        printf("Terminal control returned to parent.\n");
    }

    return 0;
}

```

Output:

```

geya@LenovoL480:~/OS/Skill/Week 9$ ./task3
Child running. PID = 1361
Giving terminal to child process...
Child completed.

[1]+  Stopped                  ./task3
geya@LenovoL480:~/OS/Skill/Week 9$ |

```

Task 4

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verifr Execution, Test Background Recovery

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>

void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received.\n");
        exit(0);
    }
}

int main()
{
    sigset_t set;

    printf("Process PID: %d\n", getpid());
    printf("Process Group ID: %d\n", getpgrp());

    /*
```

```

/*
    Register signal handlers
*/
signal(SIGINT, handle_signal);
signal(SIGTERM, handle_signal);

/*
    Create signal mask
*/
sigemptyset(&set);
sigaddset(&set, SIGINT);

printf("\nBlocking SIGINT for 5 seconds...\n");

sigprocmask(SIG_BLOCK, &set, NULL);

sleep(5);

printf("SIGINT is now unblocked.\n");

sigprocmask(SIG_UNBLOCK, &set, NULL);

printf("Try pressing Ctrl+C.\n");

while (1)
{

```

```

    {
        sleep(1);
    }

    return 0;
}
|

```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 9$ ./task4
Process PID: 1396
Process Group ID: 1396

Blocking SIGINT for 5 seconds...
SIGINT is now unblocked.
Try pressing Ctrl+C.
^C
SIGINT received.
^C
SIGINT received.
^\\Quit                               ./task4
geya@LenovoL480:~/OS/Skill/Week 9$ |
```